MISSION-CRITICAL DEPLOYMENT

HORIZON GRID

Mission-Critical Operations Manual

Version: 0.1.0

Documentation prepared: 2026-08-20

PREPARED FOR EVALUATION

Table of Contents

Mission-Critical Operations Manual

This manual is written for a HORIZON GRID deployment at a remote, physically-hard-to-reach, mission-critical site: installed once, expected to run correctly for a long time, with no developer access afterward. It documents exactly what changed to make that possible, what an on-site operator (who may have no Docker/Linux/PowerShell background) actually needs to do, and — honestly — what is still a known limitation rather than a solved problem. Where this manual and an older chapter (Operations Guide, Windows/Linux Administration) disagree on a fact like a restart policy or a health-check endpoint, **this manual is current; the older chapter predates this hardening pass and has not yet been fully rewritten.**

1. What changed, and why

A dedicated reliability review (documented in full in the project's own commit history) found and fixed the following real gaps. Each is a genuine before/after change, not a theoretical improvement:

1. **Every one of the 8 Docker Compose services now has `restart: unless-stopped`.** Previously only the two Celery containers did — a crashed backend, database, or frontend stayed down until a human ran `docker compose up`. Confirmed live: a container OOM-killed by its own memory limit now restarts automatically (`RestartCount` increments, container comes back healthy within seconds); a container an operator deliberately stops or kills stays down, matching Docker's own "operator explicitly wants this down" semantics, not a defect.
2. **A real dependency-aware health endpoint, `GET /health/detailed`**, added alongside the existing `GET /health`. The original `/health` is a pure liveness check — it returns `{"status":"ok"}` the instant the process accepts connections, even with Postgres fully stopped, and is deliberately left unchanged (CI and simple reachability checks depend on it staying dependency-free). `/health/detailed` genuinely pings Postgres and Redis and returns HTTP 503 with `"status":"down"` when Postgres is unreachable, `"status":"degraded"` (200) when only Redis is down. Every real health consumer — the Windows/Linux watchdog, `Service-Status` / `service-status.sh`, the wizard's post-install wait, and the new Docker `healthcheck:` block — was repointed at `/health/detailed`; CI's own bare-runner smoke test and simple reachability scripts stay on plain `/health`, which is their genuinely intended use.
3. **Boot-time auto-start on both platforms.** Windows: a `SYSTEM`-level Scheduled Task (`HORIZON GRID Startup`) fires once at every boot. Linux: `systemctl enable horizon-grid.service` is now actually called by the wizard — confirmed via exhaustive search that this call had never existed anywhere in the codebase, meaning the systemd unit file was correct but never actually wired to run at boot; a configured, working install would silently not come back after a host reboot.
4. **A 5-minute health watchdog on both platforms.** Windows: a second Scheduled Task running `Watchdog.ps1`. Linux: `horizon-grid-watchdog.timer` / `.service`. Both check `/health/detailed`; on failure, they log to a dedicated `watchdog.log`, run `docker compose restart`, wait 20 seconds, and log the outcome either way. A healthy check produces no log line, so `watchdog.log` only grows when there is something worth an operator's attention.
5. **Scheduled daily database backups on both platforms**, in addition to the pre-existing manual "Backup Database Now" / `horizon-grid backup` and the pre-upgrade backup the wizard already took on Windows (now added to Linux too, for parity). Windows: a third Scheduled Task at 02:00 local time. Linux: `horizon-grid-backup.timer` (`OnCalendar=*-*-* 02:00:00`, `Persistent=true` — a missed run because the machine was off catches up as soon as it's next up, rather than silently waiting a full day). Both keep the 10 most recent backups and prune older ones automatically.
6. **A real restore procedure on both platforms** (§4 below) — previously only backup scripts existed anywhere, and the one documented manual restore procedure was self-contradictory (it said to stop the platform, then restore into "the running Postgres container" — but stopping the platform also stops Postgres).
7. **SSRF guard wired into the actual AI-call path**, not just the Test-Connection convenience endpoint. The link-local-address check (blocks `169.254.0.0/16`, including every major cloud provider's instance-metadata service at `169.254.169.254`) previously ran

only when an operator clicked "Test Connection" — the real call every investigation makes was unchecked, including the common case of a wizard-driven install that never touches the web "AI Providers" panel afterward (which reads the frozen `.env`-derived setting through a completely separate code path that had no check at all). Now centralized inside the one real client-construction choke point, covering both paths.

8. **Login brute-force rate limiting** — a fixed-window limit (10 attempts / 60 seconds by default, `LOGIN_RATE_LIMIT_MAX_ATTEMPTS` / `LOGIN_RATE_LIMIT_WINDOW_SECONDS`), keyed on the attempted email, checked before password verification. Deliberately a rate limit, not a hard account lockout — a lockout would itself become a way to lock a real administrator out by deliberately failing their login.
9. **A dead-retry-code bug fixed in the provider layer.** `BaseProvider.run()` was silently catching the exact `httpx` exception types the orchestrator's retry loop was supposed to see and retry, before they could ever propagate — so `provider_max_retries` had no real effect for any provider that didn't implement its own network-error handling. Confirmed live with a fake provider: before the fix, a transient `ConnectError` was never retried; after, it is.
10. **A concurrency cap on security-assessment (port scan) execution** (`_MAX_CONCURRENT_SCANS = 4`) and **CIDR-size-proportional nmap timeout scaling** — a full /28 (16 hosts) previously got the exact same wall-clock budget as a single host.
11. **Swagger UI, ReDoc, and the raw OpenAPI schema are now disabled in production** (`ENVIRONMENT=production`, set by `docker-compose.prod.yml` — the override every real installer uses). Local development and CI's bare unit job are unaffected.
12. **A global 10 MB request-body-size limit**, plus `max_length` constraints on every previously-unbounded free-text field (investigation IOC value, case description, case note body).
13. **Both setup wizards now gate configuration save on a real Test Connection result.** Neither wizard blocks on "never tested" (a fresh install has no backend running yet to test against — this is a hard constraint, not an oversight: config is written before the stack starts), but both now block "Next" when the exact current value was just tested and failed. The install flow's own Summary/Install page additionally runs an automatic post-health-check AI connectivity test once the backend exists, reporting the honest result either way instead of silently assuming the saved configuration works.
14. **Linux's prerequisite check (`check-prerequisites.sh`) is now a real blocking gate** in the setup wizard, not an ignored informational check (the .deb's `postinst` previously discarded its exit code entirely). Only genuine hard failures (not root, Docker missing/not running, no Compose v2, insufficient disk) block; soft findings (unsupported distro, low RAM, a busy port the wizard's own Port Review page can remap) are printed as warnings only. **Found and fixed a real, previously-undiscovered bug while wiring this up:** the script's own JSON output (`{"ok": ..., "checks": [...]}`) had a permanently empty `checks` array in every real run — its `add_check()` helper interpolated bash's lowercase `true`/`false` into Python source as bare identifiers, which Python doesn't recognize as booleans, so every single call silently failed via its own error-swallowing fallback. The human-readable console output and the script's overall exit code were unaffected (computed independently in bash), which is exactly why this went unnoticed until something finally tried to consume the JSON programmatically.
15. **Redis now has a persistent volume.** Confirmed live before this fix: a value set, then a container restart (an image update, a reconfigure), silently reset it to empty. Nothing stored there is a system of record (provider-result cache, rate-limiter counters, the Celery broker), but a routine reconfigure resetting every rate limiter and the whole cache was still a real, avoidable behavior change.
16. **The AI-outcome badge on the Final Assessment panel now distinguishes a genuine AI failure from a correct "nothing to assess" decision.** Both previously rendered the identical "No AI call (no evidence to assess)" text — the backend already tracked which case applied (`ai_outcome`: `success` / `failed` / `skipped_no_evidence`) and sent it in every response, but the frontend's type didn't declare the field and no component read it. Confirmed live against a real `ai_outcome="failed"` row that the backend was already sending the field correctly — this was a pure frontend gap.

2. Zero-hands operation: what now happens automatically, and what still needs a human

Fully automatic, no operator action required, once configured:

- The platform starts itself after any host reboot (including an unattended one after a power outage).
- A crashed or OOM-killed container restarts itself.

- An unhealthy-but-still-running backend (containers up, application inside genuinely broken) is detected and restarted within 5 minutes.
- The database is backed up every night; the 10 most recent snapshots are always kept.

Still requires a human, by design (not an oversight):

- **Restoring a backup.** Deliberately not automated — an automatic restore is one of the more dangerous things a system could silently decide to do on its own. Run `Restore Database` (Windows Start Menu) or `sudo horizon-grid restore` (Linux); both require an explicit typed confirmation (`RESTORE`) unless `-Force` / `--force` is passed, since this permanently discards everything written since the backup being restored.
- **Changing configuration** (AI backend, provider keys, ports, administrator password) — via the Configuration wizard / `horizon-grid configure`, or the web UI's own Administration pages once signed in.
- **Recovering from a genuinely exhausted disk.** Log rotation (10 MB × 3 files per service) and backup pruning bound *this platform's own* growth, but do not free space consumed by something else on the machine.
- **Applying a new installer/version.** This is a deliberate, reviewed action (see the pre-upgrade backup in §1.5), not a silent background update.

3. Health monitoring, precisely

Check	Endpoint / command	What it actually proves	Who calls it
Liveness	`GET /health`	The backend process is accepting connections. Nothing about Postgres/Redis.	CI's bare smoke test, simple uptime pings
Dependency health	`GET /health/detailed`	Postgres is genuinely reachable (503 if not); Redis reachability is reported but only degrades status, doesn't fail it	The watchdog (both platforms), `Service-Status` / `service-status.sh`, the Docker `healthcheck:` block, the wizard's post-install wait
Container-level	`docker compose ps`	Whether Docker itself thinks each container is running/healthy — does not prove the *application* inside is working	Service Status shortcut, `horizon-grid status`
Operational	`GET /api/v1/dashboard/kpis`, `GET /api/v1/providers/health`	Whether real investigations are succeeding, not just whether the process is up	The web dashboard

`/health` and `/health/detailed` both report `version` and `uptime_seconds` — "is it healthy, and what version/uptime is it on" is answerable in one request, in well under a second, matching the requirement that remote-site status be checkable quickly without a developer present.

4. Backup and restore, step by step

Taking a backup manually: Windows Start Menu → **Backup Database Now**, or `sudo horizon-grid backup`. Both run `pg_dump` inside the live Postgres container (no separate `psql` / `pg_dump` install needed on the host) and prune to the 10 most recent snapshots automatically.

Restoring a backup:

1. Windows Start Menu → **Restore Database** (or `Restore-Database.ps1 -BackupFile <path> [-Force]`), or `sudo horizon-grid restore [/path/to/backup.sql] [--force]`. Omitting the file argument restores the most recent backup found.
2. Without `-Force` / `--force`, you are shown the exact file that will be used and must type `RESTORE` (capitals) to proceed. This is not a soft confirmation — anything else cancels with no changes made.
3. What happens next, in order: the services that write to the database (`backend`, `celery_worker`, `celery_beat`) are stopped — Postgres itself is deliberately left running, since the restore needs a live server to connect to; any other open connections to the database are terminated; the target database is dropped and recreated (required because a plain `pg_dump` with no `--clean` cannot be replayed on top of already-existing data); the backup file is replayed; the stopped services are restarted; the script waits up to 90 seconds for `/health/detailed` to report healthy.
4. This procedure was verified end-to-end on a real installation during this hardening review: a backup was taken, a marker row was inserted, the restore was run, and the marker row was confirmed gone while the platform came back up correctly.

What is and isn't covered by a backup: the Postgres database (all investigations, cases, users, runtime provider/AI configuration) is what `pg_dump` captures. It does **not** include `.env` (API keys, JWT secret, database credentials — back this up separately, e.g. by copying `config\.env` / `/etc/horizon-grid/.env` to a secure location) or the Redis cache (deliberately not a system of record — see §1.15).

5. Disaster-recovery scenarios, and what to actually do

- **Unattended host reboot (e.g. after a power outage).** No action needed — boot-time auto-start (§1.3) brings the stack back up on its own. Verify with `Service Status` / `horizon-grid status` once reachable.
- **Backend process becomes unresponsive but the container is still "running."** No action needed for the first 5 minutes — the watchdog (§1.4) detects and restarts it automatically. If `watchdog.log` shows repeated failed recovery attempts, this indicates a persistent problem (e.g. a corrupted database) that a restart alone cannot fix — proceed to the restore procedure (§4) or Diagnostics.
- **Database corruption or a bad configuration change.** Restore the most recent known-good backup (§4). If no backup exists yet (a fresh install less than a day old, before the first scheduled 02:00 backup has run), there is nothing to restore from — this is the concrete reason the pre-upgrade backup and the daily schedule both exist.
- **Forgotten administrator password / all administrators disabled.** There is deliberately no insecure universal backdoor. Recovery requires direct database access (an administrator with host/root access can update `users.hashed_password` directly, or restore a backup from before the lockout) — this is a known, disclosed limitation of a system with no remote support channel, not an unaddressed gap; building a passwordless backdoor would itself be a worse security posture for a mission-critical, physically-inaccessible deployment.
- **Disk approaching full.** Log rotation and backup pruning bound this platform's own footprint; there is currently no automated alert when the underlying disk itself is low (see §6, Known Limitations).

6. Security hardening summary

- Outbound SSRF guard on the Ollama AI-call path (§1.7) — blocks link-local addresses including cloud instance-metadata services, on both the runtime-config and `.env`-only paths.
- Login brute-force rate limiting (§1.8), fixed-window per attempted email.
- Swagger UI / ReDoc / raw OpenAPI schema disabled in production (§1.11).
- Global request-body-size limit (10 MB) and `max_length` on every free-text field (§1.12).
- Every credential (API keys, database password, JWT signing secret) lives only in a root/Administrator-locked `.env` file, never logged or included in the Diagnostics bundle (which redacts to `***REDACTED***(set, N chars)`).

- Nmap (the port-scanning tool) is invoked via a hardcoded argument list per profile, never a shell, never with a caller-supplied flag — see the Security Assessment Toolkit chapter for the full design.
- A **known, disclosed, not-yet-closed gap**: the SSRF check validates a DNS resolution snapshot; the real outbound HTTP call re-resolves independently afterward, leaving a narrow DNS-rebinding TOCTOU window. Not fixed in this pass — disclosed here rather than silently left undocumented.

7. Known limitations (disclosed, not fixed, in this hardening pass)

Per this project's own certification standard — certify only what was actually implemented and verified, not what was intended — the following are real, identified gaps that remain open:

- **No automated disk-space alerting.** Log rotation and backup pruning bound this platform's own growth; nothing alerts an operator if the underlying host disk itself is running low.
- **No retention/cleanup job for ever-growing investigation tables** (`ioc_lookups`, `provider_results`, `evidence_items`, `config_audit_log`, and others) — data accumulates indefinitely.
- **Neo4j and OpenSearch are fully provisioned (roughly 1.5–2 GB RAM reserved) with zero actual read/write traffic from the application**, confirmed via exhaustive code search. This is a real resource cost on a remote/constrained site with no corresponding current benefit — an open question for the product's direction, not something this review resolved unilaterally.
- **The DNS-rebinding TOCTOU gap** described in §6.
- **No off-host/off-site backup copy option** — backups are always written to local disk only. For a genuinely remote, physically-inaccessible site, a local-disk-only backup does not protect against the disk/host itself failing.
- **A full multi-hour soak/long-run test's results are reported separately** (see the release-gate certification report) rather than claimed here in advance of that run completing.

8. Where to look next

- **Windows/Linux Administration** chapters — day-to-day shortcuts and commands (superseded on restart-policy and health-endpoint specifics by §1 and §3 of this manual, otherwise still accurate).
- **Operations Guide** — Celery/cache internals, deployment topology detail.
- **Security Assessment Toolkit** chapter — the port-scanning module's full safety design.
- **Deployment and Troubleshooting** chapter — the complete environment-variable reference.